FinPilot — System Design Interview Document

> ***Time Budget:*** *10 minutes*
> ***Format:*** *Low-level design walkthrough suitable for a system design interview*
> ***Audience:*** *Senior engineers / hiring panel*

Table of Contents

---

1. Elevator Pitch

> *FinPilot is a full-stack personal finance application that lets users track multi-currency transactions, set per-category monthly budgets, import bank data via CSV, and get AI-powered savings recommendations. It's built as two separate deployments — a React SPA on CloudFront/S3 and a Node.js REST API on ECS Fargate backed by Aurora PostgreSQL Serverless. For financial accuracy, currency conversion is locked at write-time so historical reports never change retroactively.*

---

2. Functional Requirements

Feature	Description
Auth	Email/password registration + Google OAuth. JWT in HttpOnly cookies (access: 15 min, refresh: 7 days).
Transactions	Full CRUD with per-user scoping. Multi-currency with write-time base-currency conversion. Pagination, filtering by date/type/category/amount range.
CSV Import	Upload `.csv` (max 2 MB), validate row-by-row (invalid rows don't block valid ones), auto-category matching, audit trail via `BankSyncLog`.
Categories	2-level hierarchy (parent → child) via self-referencing FK. Typed as INCOME or EXPENSE. Default categories seeded on registration.
Budgets	Per-category per-month limits with configurable alert thresholds (default 90%). Budget vs. actual comparison.

Dashboard	Computed on-demand: income/expense totals, monthly trend, category breakdown, recent transactions — all from PostgreSQL aggregates.
AI Savings Planner	6-month spending analysis → pre-aggregated monthly summaries → deterministic trend detection + LLM call → per-category savings recommendations.
Reports	PDF and CSV export for any date range. Scheduled email reports via cron.
Bank Sync	Provider abstraction pattern (`BankProvider` interface). Current: MockProvider + CSV. Ready for Plaid/TrueLayer by implementing the interface.
Audit Log	Append-only log of all mutations: entity type, entity ID, action, old/new values, IP address.

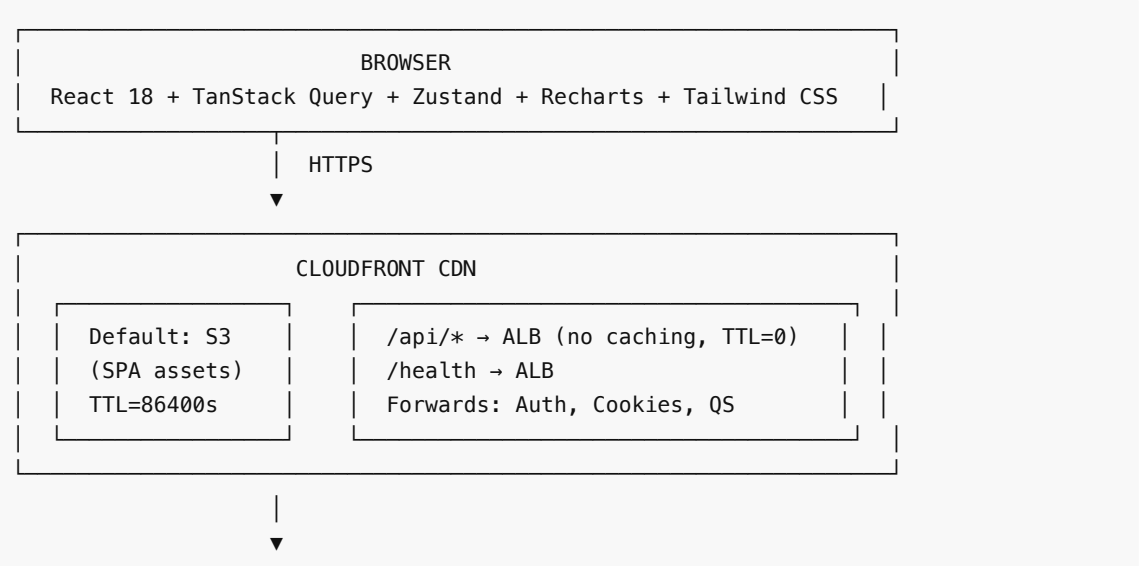
---

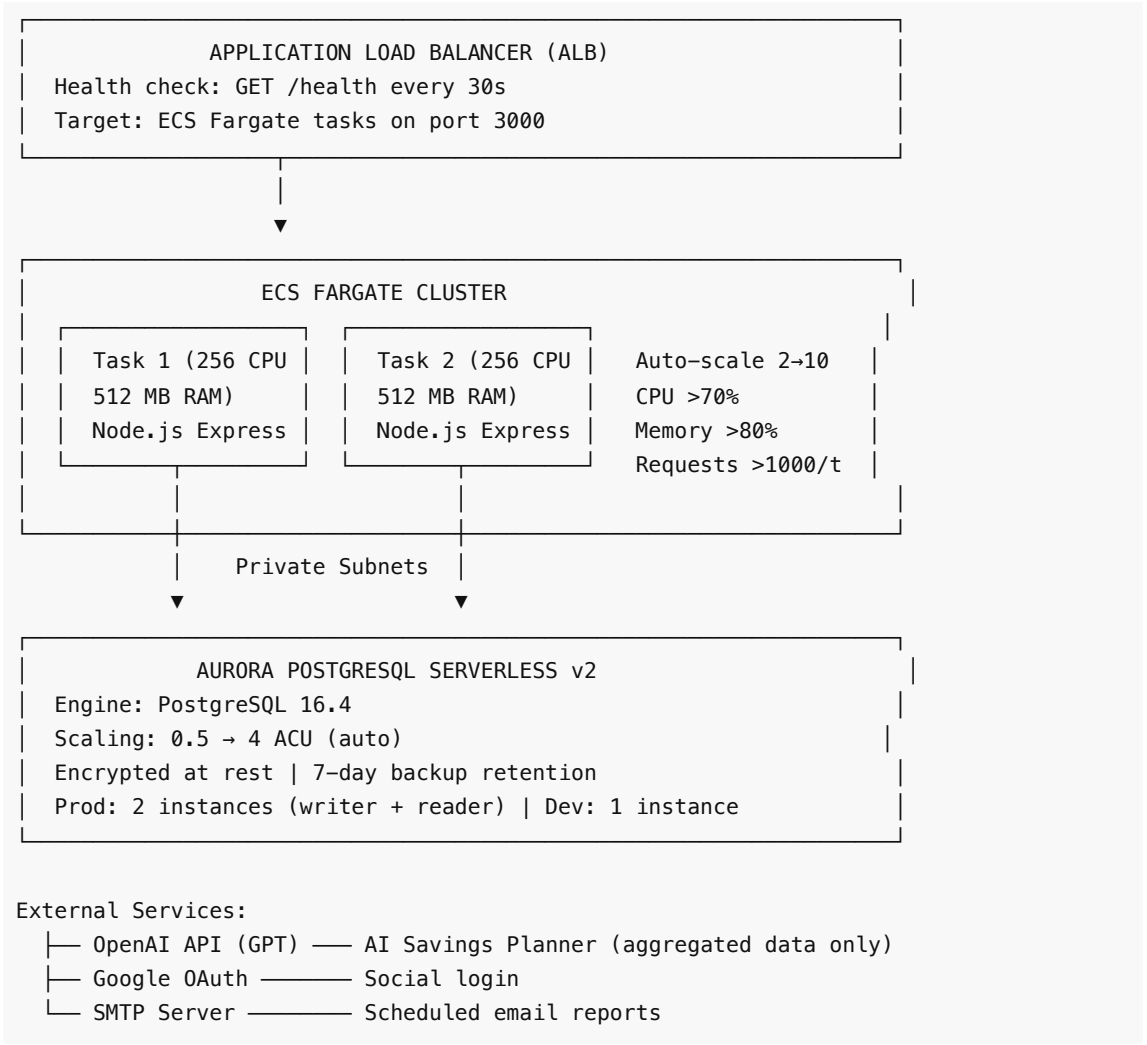
3. Non-Functional Requirements

Requirement	Target
Latency	p95 < 300 ms for dashboard queries (achieved at low concurrency)
Availability	99.9% (Multi-AZ Aurora, ALB health checks, ECS deployment circuit breakers)
Consistency	Strong consistency — no eventual consistency trade-offs. Monetary data must be deterministic.
Concurrency	~5–10 concurrent users on current 0.25 vCPU config; scales to 50+ at 1 vCPU (identified via stress testing)
Security	OWASP Top 10 mitigated: HttpOnly cookies, bcrypt, Zod validation, parameterized queries (Prisma), CORS, no raw SQL
Data Privacy	AI never sees raw transactions, only pre-aggregated monthly category summaries

---

4. High-Level Architecture

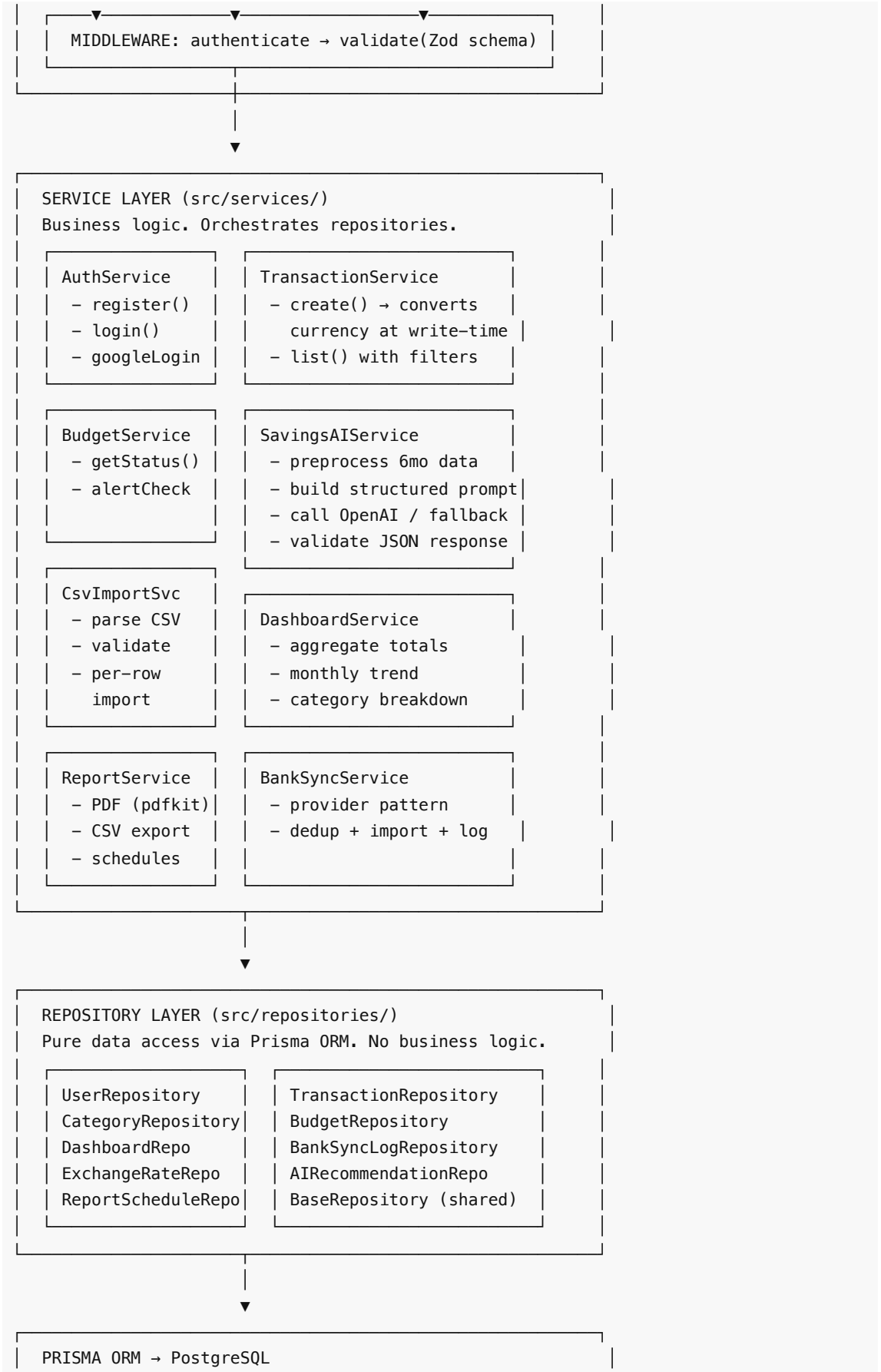




---

5. Low-Level Design

5.1 Backend Layer Architecture


MIDDLEWARE: authenticate → validate(Zod schema)
SERVICE LAYER (src/services/)
Business logic. Orchestrates repositories.
AuthService
- register()
- login()
- googleLogin
TransactionService
- create() → converts currency at write-time
- list() with filters
BudgetService
- getStatus()
- alertCheck
SavingsAIService
- preprocess 6mo data
- build structured prompt
- call OpenAI / fallback
- validate JSON response
CsvImportSvc
- parse CSV
- validate
- per-row import
DashboardService
- aggregate totals
- monthly trend
- category breakdown
ReportService
- PDF (pdfkit)
- CSV export
- schedules
BankSyncService
- provider pattern
- dedup + import + log
REPOSITORY LAYER (src/repositories/)
Pure data access via Prisma ORM. No business logic.
UserRepository
CategoryRepository
DashboardRepo
ExchangeRateRepo
ReportScheduleRepo
TransactionRepository
BudgetRepository
BankSyncLogRepository
AIRecommendationRepo
BaseRepository (shared)
PRISMA ORM → PostgreSQL

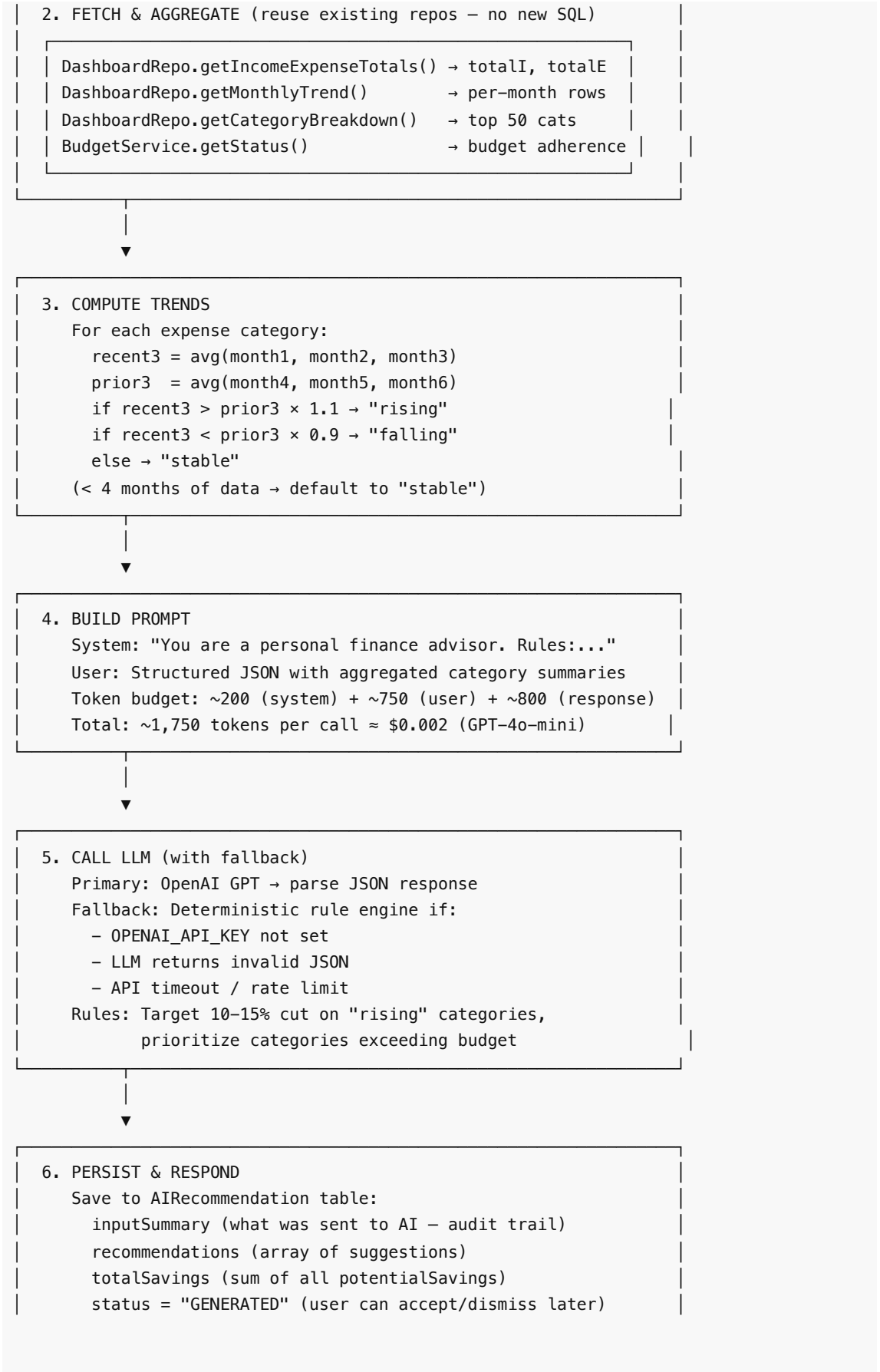
```
│  Type-safe queries, parameterized (SQL injection safe)    │
│  Singleton client via database.ts                         │
└──────────────────────────────────────────────────────────┘
```

5.2 Frontend Layer Architecture

```
┌──────────────────────────────────────────────────────────┐
│  PAGES (src/pages/)                                       │
│  One component per route. Composes smaller components.    │
│  Dashboard | Transactions | Categories | Budgets |        │
│  Reports | Settings | Login | Register                    │
└──────────────────────┬───────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────┐
│  COMPONENTS (src/components/)                             │
│  Reusable UI: forms, tables, charts, modals, cards        │
│  TransactionForm | BudgetCard | CategoryTree |            │
│  DashboardCharts (Recharts) | FilterBar                   │
└──────────────────────┬───────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────┐
│  HOOKS (src/hooks/)                                       │
│  Wrap TanStack Query mutations/queries.                   │
│  Components NEVER call APIs directly.                     │
│  useTransactions | useBudgets | useDashboard | useAuth    │
└──────────────────────┬───────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────┐
│  API CLIENT (src/lib/api.ts)                              │
│  Axios instance with JWT cookie interceptor.              │
│  Base URL: /api (proxied via CloudFront → ALB)            │
└──────────────────────────────────────────────────────────┘
```

5.3 AI Savings Planner Pipeline (Detailed)

```
┌──────────────────────────────────────────────────────────────┐
│  GET /api/ai/recommendations                                  │
└──────────────────────┬───────────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────────┐
│  1. DETERMINE WINDOW                                          │
│     end = last day of current month                           │
│     start = 1st day of (current month − 5) → 6-month window   │
└──────────────────────┬───────────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────────┐
```


2. FETCH & AGGREGATE (reuse existing repos — no new SQL)
DashboardRepo.getIncomeExpenseTotals() → totalI, totalE
DashboardRepo.getMonthlyTrend() → per-month rows
DashboardRepo.getCategoryBreakdown() → top 50 cats
BudgetService.getStatus() → budget adherence
3. COMPUTE TRENDS
For each expense category:
recent3 = avg(month1, month2, month3)
prior3 = avg(month4, month5, month6)
if recent3 > prior3 × 1.1 → "rising"
if recent3 < prior3 × 0.9 → "falling"
else → "stable"
(< 4 months of data → default to "stable")
4. BUILD PROMPT
System: "You are a personal finance advisor. Rules:..."
User: Structured JSON with aggregated category summaries
Token budget: ~200 (system) + ~750 (user) + ~800 (response)
Total: ~1,750 tokens per call ≈ $0.002 (GPT-4o-mini)
5. CALL LLM (with fallback)
Primary: OpenAI GPT → parse JSON response
Fallback: Deterministic rule engine if:
- OPENAI_API_KEY not set
- LLM returns invalid JSON
- API timeout / rate limit
Rules: Target 10-15% cut on "rising" categories,
prioritize categories exceeding budget
6. PERSIST & RESPOND
Save to AIRecommendation table:
inputSummary (what was sent to AI — audit trail)
recommendations (array of suggestions)
totalSavings (sum of all potentialSavings)
status = "GENERATED" (user can accept/dismiss later)

```
│     Return recommendations to frontend                          │
└─────────────────────────────────────────────────────────────────┘
```

5.4 CSV Import Flow (Detailed)

```
POST /api/transactions/import
│  multipart/form-data (max 2 MB)
│
├── Multer middleware: memory storage, size limit enforced
│
├── Validation: file exists? .csv extension?
│
├── CsvImportService.importTransactions(userId, buffer)
│   │
│   ├── 1. Create BankSyncLog (status: PENDING, source: CSV)
│   │
│   ├── 2. Parse CSV buffer with csv-parser
│   │      Required columns: date, amount, type, category
│   │      Optional: description, currency
│   │
│   ├── 3. For each row (independent — bad rows don't block good ones):
│   │   ├── Validate date (parseable), amount (> 0), type (INCOME|EXPENSE)
│   │   ├── Resolve category by name (case-insensitive, must exist)
│   │   ├── Verify type ↔ category consistency
│   │   ├── Lookup exchange rate if currency ≠ baseCurrency
│   │   ├── Create Transaction with bankSyncLogId FK
│   │   └── On failure: record error, continue to next row
│   │
│   ├── 4. Update BankSyncLog: status (SUCCESS|PARTIAL|FAILURE),
│   │      transactionCount, errorMessage
│   │
│   └── 5. Return { imported: N, failed: M, errors: [...] }
```

5.5 Write-Time Currency Conversion

```
User creates transaction: { amount: 100, currency: "EUR" }
User's baseCurrency: "USD"
                    │
                    ▼
┌──────────────────────────────────────────────────────────┐
│  CurrencyService.convertToBase(100, "EUR", "USD")        │
│  │                                                       │
│  ├── Lookup ExchangeRate table: EUR → USD rate = 1.0850  │
│  ├── baseCurrencyAmount = 100 × 1.0850 = 108.50          │
│  └── Return { baseCurrencyAmount: 108.50, rate: 1.0850 } │
└──────────────────────────────────────────────────────────┘
                    │
                    ▼
  Store in Transaction:
    amount           = 100.00    (original)
    currency         = "EUR"     (original)
```

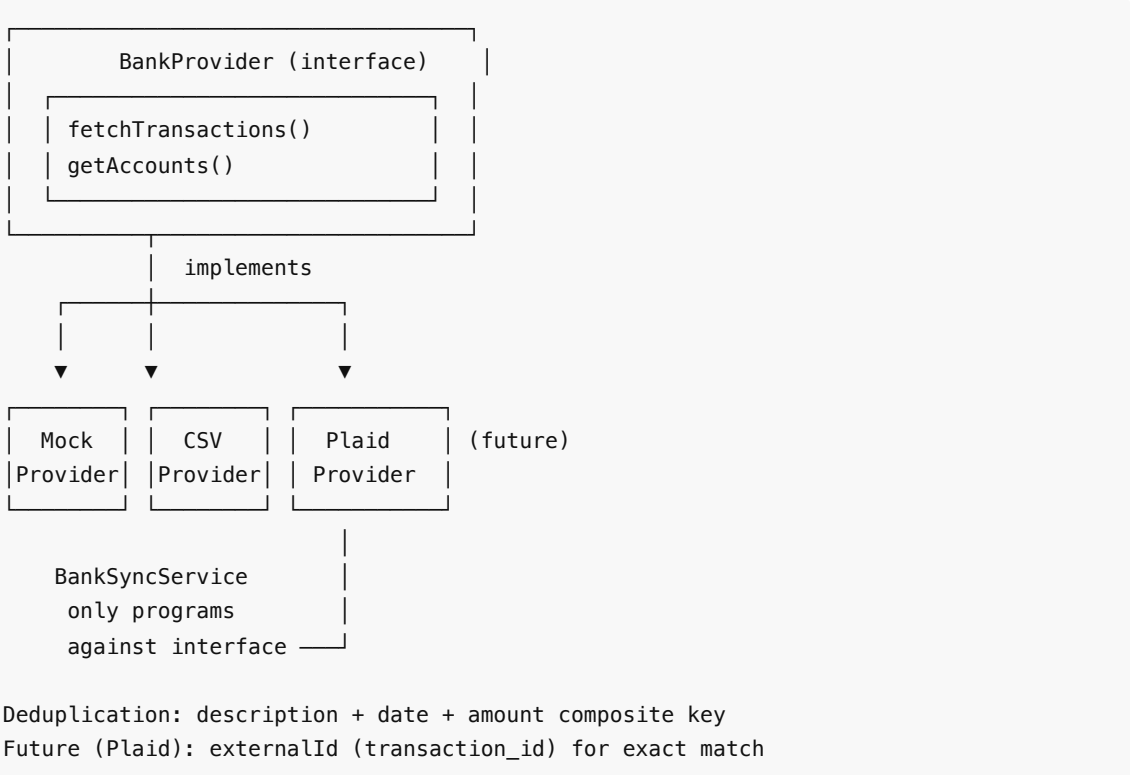
```
    baseCurrencyAmount = 108.50  (converted, IMMUTABLE)
    exchangeRate     = 1.085000  (locked, IMMUTABLE)
```

Why write-time, not read-time?

Concern	Write-Time (our choice)	Read-Time (alternative)
Query speed	Fast — aggregate `baseCurrencyAmount` directly	Slow — join ExchangeRate for every row
Historical accuracy	Locked to rate when event occurred	Retroactively changes past totals as rates fluctuate
Audit trail	Rate is immutable evidence	No record of what rate applied
Storage cost	+2 columns per transaction	No extra columns

Verdict: Financial apps need deterministic totals. A March report viewed in March and again in June must show the same numbers. Storage overhead is negligible.

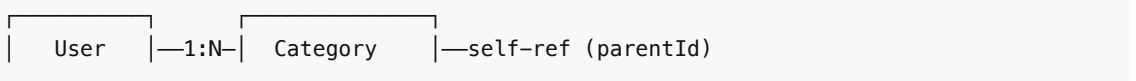
5.6 Bank Sync Provider Pattern

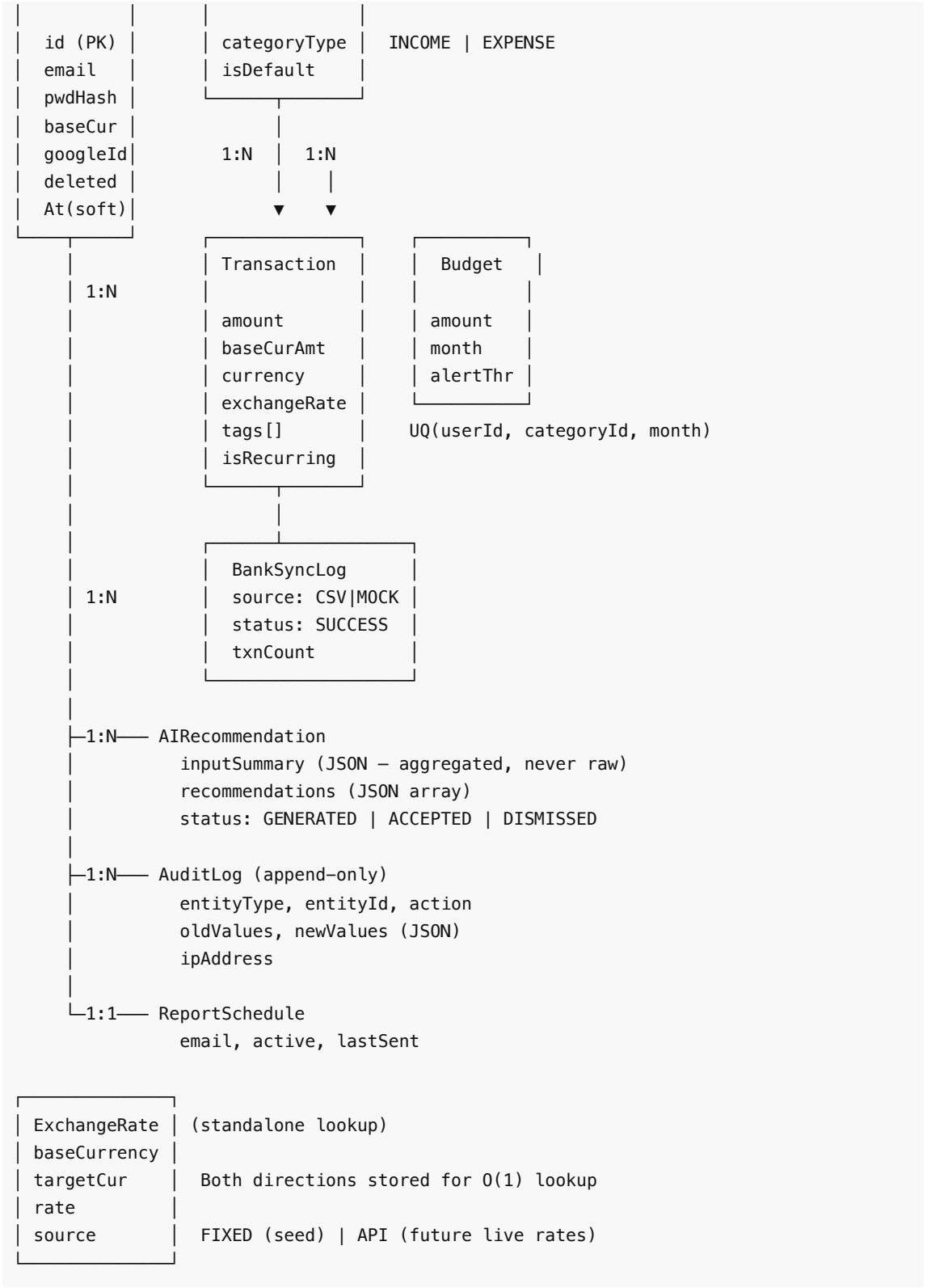


---

6. Database Design

6.1 Entity-Relationship Diagram





6.2 Key Design Decisions

Decision	Rationale

CUID primary keys	URL-safe, sortable, no sequential enumeration risk (unlike auto-increment IDs that leak record count)
Decimal(15,2) for money	Avoids floating-point errors. 15 digits supports up to trillions.
Decimal(10,6) for FX rates	6 decimal places for precise forex conversions
Soft-delete on User only	`deletedAt` preserves referential integrity. Transactions/categories cascade on hard delete.
JSON fields for AI & metadata	`inputSummary`, `recommendations`, `metadata` — flexible schema that evolves without migrations
Both FX directions stored	USD→EUR and EUR→USD both in table. O(1) lookup vs. computing inverse (precision loss).
Category self-relation	`parentId` FK to same table. Simple 2-level hierarchy without junction table.
Composite indexes	Every dashboard/list query has a covering index (see below)

6.3 Index Strategy

```
-- Transaction (the most queried table)
@@index([userId, transactionDate])                    -- date-range listing
@@index([userId, categoryId])                         -- category views
@@index([userId, transactionType])                    -- income/expense filter
@@index([userId, transactionType, transactionDate])   -- dashboard totals
@@index([userId, categoryId, transactionDate])        -- budget-vs-actual
@@index([bankSyncLogId])                              -- import batch lookup

-- Budget
@@unique([userId, categoryId, month])                 -- one budget per cat/month
@@index([userId, month])                              -- monthly budget list

-- Category
@@unique([userId, name, parentId])                    -- no duplicate names at same
level
@@index([userId, categoryType])                       -- filter by type

-- AuditLog
@@index([userId, entityType])                         -- user's audit history
@@index([entityType, entityId])                       -- all actions on a record
@@index([createdAt])                                  -- time-range queries

-- BankSyncLog
@@index([userId, status])                             -- sync history
@@index([userId, createdAt])                          -- recent syncs
```

---

7. API Design

7.1 Complete Endpoint Map (39 endpoints)

Method	Endpoint	Auth	Purpose
POST	/api/auth/register	No	Create account
POST	/api/auth/login	No	Email/password login
POST	/api/auth/google	No	Google OAuth login
GET	/api/auth/google-client-id	No	Fetch Google client ID
POST	/api/auth/logout	No	Clear cookies
GET	/api/auth/me	No*	Current user profile (returns null, not 401)
GET	/api/transactions	Yes	List with filters + pagination
GET	/api/transactions/:id	Yes	Get single transaction
POST	/api/transactions	Yes	Create transaction
POST	/api/transactions/import	Yes	CSV file upload
PATCH	/api/transactions/:id	Yes	Update transaction
DELETE	/api/transactions/:id	Yes	Delete transaction
GET	/api/categories	Yes	List all user categories
GET	/api/categories/:id	Yes	Get single category
POST	/api/categories	Yes	Create category
PATCH	/api/categories/:id	Yes	Update category
DELETE	/api/categories/:id	Yes	Delete category
GET	/api/budgets?month=	Yes	List budgets for month
GET	/api/budgets/status?month=	Yes	Budget vs actual
POST	/api/budgets	Yes	Create budget
PATCH	/api/budgets/:id	Yes	Update budget
DELETE	/api/budgets/:id	Yes	Delete budget
GET	/api/dashboard?startDate=&endDate=	Yes	Full dashboard summary
GET	/api/reports/pdf?startDate=&endDate=	Yes	Download PDF report
GET	/api/reports/csv?startDate=&endDate=	Yes	Download CSV export

GET	/api/reports/schedule	Yes	Get report schedule
POST	/api/reports/schedule	Yes	Create/update schedule
DELETE	/api/reports/schedule	Yes	Delete schedule
POST	/api/sync/bank	Yes	Trigger bank sync
GET	/api/sync/history	Yes	Sync audit trail
GET	/api/sync/providers	Yes	Available providers
GET	/api/ai/recommendations	Yes	Get AI savings plan
POST	/api/ai/recommendations/:id/status	Yes	Accept/dismiss recommendation
GET	/api/ai/recommendations/history	Yes	Past recommendations
GET	/health	No	Health check (not under /api)

7.2 Authentication Flow

```
Register/Login
    │
    ├── Server returns two HttpOnly cookies:
    │   ├── accessToken  (JWT, 15 min expiry)
    │   └── refreshToken (JWT, 7 day expiry)
    │
    ├── Every authenticated request:
    │   Browser auto-sends cookies → auth middleware extracts accessToken
    │   → jwt.verify() → attaches req.user = { userId }
    │
    └── Why HttpOnly cookies over Authorization header?
        ├── XSS-resistant (JavaScript cannot read HttpOnly cookies)
        ├── CORS-friendly for separate frontend/backend domains
        ├── Stateless — no server-side session store needed
        └── Scales horizontally — any ECS task can verify the JWT
```

7.3 Response Format (consistent contract)

```
// Success
{
  "success": true,
  "data": { ... },
  "meta": { "page": 1, "limit": 20, "total": 150, "totalPages": 8 }
}

// Error
{
  "success": false,
  "message": "Category not found",
```

```
  "errors": [{ "field": "categoryId", "message": "Invalid ID" }]
}
```

---

8. Network Bandwidth Assumptions

8.1 Per-Request Payload Sizes

Operation	Request Size	Response Size	Notes
Login/Register	~200 B	~500 B	Password + email in; JWT in cookie, user profile out
Create Transaction	~300 B	~400 B	JSON body in; created object out
List Transactions (page=20)	~100 B (query string)	~8 KB	20 transactions × ~400 B each
Dashboard Summary	~100 B	~5 KB	Totals + monthly trend (12 points) + top categories
CSV Import (max)	**2 MB** (file limit)	~2 KB	File upload; summary response
AI Recommendations	~100 B	~3 KB	Aggregated prompt ~2 KB internal, response ~3 KB
PDF Report Download	~100 B	**50–200 KB**	Depends on transaction count
CSV Export	~100 B	**10–500 KB**	Raw transaction data
Frontend SPA (initial)	—	**~500 KB** (gzipped)	React bundle + assets (CloudFront cached, TTL=1 year)

8.2 Bandwidth Estimation (1,000 DAU)

Assumptions:

- 1,000 daily active users
- Average session: 5 minutes, ~15 API calls
- 10% of users import CSV per day
- 5% of users download reports per day
- 2% of users use AI recommendations per day

Traffic Type	Calculation	Daily Bandwidth
API Reads	1,000 users × 12 reads × ~5 KB avg	**~60 MB/day**
API Writes	1,000 users × 3 writes × ~300 B	**~0.9 MB/day**
CSV Imports	100 users × 500 KB avg file	**~50 MB/day**
Report Downloads	50 users × 150 KB avg PDF	**~7.5 MB/day**
AI Calls	20 users × 3 KB response	**~0.06 MB/day**

SPA Assets	1,000 users × 500 KB (first visit, CDN cacheable)	**~500 MB/day** (peak, mostly cached)
Total		**~620 MB/day**

8.3 CDN Offloading

- Static assets (JS/CSS/images): **99%+ served from CloudFront edge** (TTL = 1 year for hashed assets)
- API calls: **0% cached** (TTL = 0, all forwarded to ALB) — correct for personalized financial data
- Net backend bandwidth: **~120 MB/day** (API traffic only)

8.4 OpenAI API Token Usage

Component	Tokens
System prompt (fixed)	~200
User message (15 categories)	~750
Completion (max)	~800
Total per call	**~1,750**
Cost per call (GPT-4o-mini)	**~$0.002**
Daily cost (20 users)	**~$0.04**
Monthly cost (1,000 DAU × 2% use)	**~$1.20**

---

9. Database Storage Estimation & Clearing Strategy

9.1 Per-Table Row Size Estimates

Table	Row Size (avg)	Notes
User	~300 B	Email, hash, name, currency, timestamps
Category	~200 B	Name, type, color, icon, parent FK
Transaction	~400 B	Amount fields, description, tags array, dates, FKs
Budget	~150 B	Amount, month, threshold, FKs
ExchangeRate	~100 B	Currency pair + rate
BankSyncLog	~500 B	Metadata JSON can vary
AIRecommendation	~2 KB	inputSummary + recommendations JSON
AuditLog	~1 KB	oldValues/newValues JSON snapshots
ReportSchedule	~150 B	Email, active flag, timestamps

9.2 Storage Growth Projection (1,000 users)

Assumptions per user per month:

- 60 transactions (2/day average)
- 3 budgets
- 2 CSV imports
- 1 AI recommendation call
- ~100 audit log entries (creates + updates)

Table	Rows/Month (1K users)	Size/Month	1 Year
Transaction	60,000	~24 MB	**~288 MB**
AuditLog	100,000	~100 MB	**~1.2 GB**
AIRecommendation	1,000	~2 MB	**~24 MB**
BankSyncLog	2,000	~1 MB	**~12 MB**
Budget	3,000	~0.5 MB	**~6 MB**
Category	~200 (mostly one-time)	~40 KB	**~40 KB**
ExchangeRate	~42 (7 currencies × 6 pairs)	~4 KB	**~4 KB**
Total		**~128 MB/month**	**~1.53 GB/year**

9.3 Index Overhead

Indexes typically add **30–50%** to table storage:

- Transaction (6 indexes) on 288 MB data → ~430 MB total with indexes after 1 year
- AuditLog (3 indexes) on 1.2 GB data → ~1.6 GB total with indexes after 1 year
- **Total with indexes: ~2.2 GB/year for 1,000 users**

9.4 Aurora Serverless v2 Capacity

- **Min capacity:** 0.5 ACU (~1 GB RAM, ~$43/month)
- **Max capacity:** 4 ACU (~8 GB RAM, ~$345/month)
- **Storage:** Auto-scales, billed per GB-month ($0.10/GB)
- At 2.2 GB/year for 1K users → **$0.22/month** in storage costs — negligible
- Storage won't be the bottleneck until ~100K+ active users

9.5 Data Clearing & Retention Strategy

```
┌──────────────────────────────────────────────────────────────┐
│                    DATA LIFECYCLE TIERS                      │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  HOT DATA (keep indefinitely in primary DB)                  │
│  ├── Users, Categories, Budgets, ExchangeRates               │
│  ├── Transactions (last 2 years)                             │
│  └── ReportSchedule                                          │
│                                                              │
│  WARM DATA (archive after 2 years)                           │
│  ├── Transactions older than 2 years                         │
│  │   → Move to partitioned archive table or S3 Parquet       │
```

```
│  │   → Still queryable via reports but not in daily dashboard   │
│  └── AIRecommendation older than 1 year                         │
│      → Aggregate into historical summary, delete raw records    │
│                                                                 │
│  COLD DATA (purge on schedule)                                  │
│  ├── AuditLog older than 1 year                                 │
│  │   → Export to S3 (compressed JSON) for compliance            │
│  │   → Delete from primary DB                                   │
│  │   → Reason: AuditLog is 78% of storage growth                │
│  ├── BankSyncLog older than 6 months (status = SUCCESS)         │
│  │   → Keep FAILURE/PARTIAL logs for 1 year for debugging       │
│  └── Soft-deleted Users after 90-day grace period               │
│      → Hard-delete cascades all child records                   │
│                                                                 │
│  CLEARING SCHEDULE (implemented via node-cron)                  │
│  ├── Daily:  None (no hot data expires daily)                   │
│  ├── Weekly: Purge expired BankSyncLog entries                  │
│  └── Monthly: Archive old AuditLogs to S3, trim AI history      │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Why AuditLog is the #1 target for clearing:

- Grows at ~100 KB/user/month (vs 24 KB for transactions)
- Represents **~78%** of annual storage growth
- After compliance review period (1 year), raw audit data has diminishing value
- Compressed export to S3 reduces cost by ~10x

---

10. Technology Choices & Why

10.1 Backend Stack

Choice	Alternative Considered	Why This One
Node.js + Express	Fastify, NestJS, Go, Python/Django	Express is the most widely adopted Node.js framework. Simple, minimal overhead, massive ecosystem. NestJS adds Angular-style DI which is over-engineered for this scope. Go would be faster but slower iteration speed for a full-stack solo project.
TypeScript	Plain JavaScript	Non-negotiable for a financial app. Compile-time type safety catches entire categories of bugs (wrong amount types, missing fields). Prisma generates type-safe DB queries.
Prisma ORM	TypeORM, Knex, Drizzle, raw SQL	Type-safe queries generated from schema (no runtime SQL string building → SQL injection safe by design). Schema-as-code with migration support. Auto-generated TypeScript types match DB perfectly. Downside: slower than raw SQL for complex analytics queries, but acceptable for OLTP workload.

PostgreSQL	MySQL, MongoDB, DynamoDB	ACID compliance is critical for financial data. `Decimal` type for exact monetary arithmetic. JSON columns for flexible AI/metadata storage. Array type for tags. Rich aggregate functions for dashboard queries.
Zod	Joi, Yup, class-validator	Runtime schema validation that infers TypeScript types. Lightweight, composable, zero dependencies. Used at API boundary for request validation (defense-in-depth with Prisma's type safety).
bcrypt	argon2, scrypt	Industry standard for password hashing. 12 salt rounds = ~250ms on full CPU. Trade-off: CPU-intensive (stress test bottleneck at 0.25 vCPU), but correct security choice. Would not weaken rounds to gain perf — would scale CPU instead.
JWT (HttpOnly cookies)	Session cookies + Redis, Passport.js	Stateless = no session store needed = horizontal scaling. HttpOnly = XSS protection. 15-min access + 7-day refresh = good balance of security and UX. Passport.js adds unnecessary abstraction for just email + Google auth.
Winston	Pino, Bunyan, console.log	Structured JSON logging with transport support (stdout → CloudWatch). Log levels (info/warn/error). Production-ready.
PDFKit	Puppeteer, jsPDF	Server-side PDF generation without a headless browser. Lightweight, no Chrome dependency. Sufficient for financial report tables.
node-cron	AWS EventBridge, Bull/BullMQ	In-process scheduler for MVP. Runs report email cron. Simple, no external infra dependency. At scale: would migrate to EventBridge for reliability and decoupling.
multer	busboy, formidable	Standard Express file upload middleware. Memory storage (no temp files on disk) → works with Fargate ephemeral storage. 2 MB limit enforced at middleware level.

10.2 Frontend Stack

Choice	Alternative Considered	Why This One
React 18	Vue, Svelte, Angular	Largest ecosystem, hiring pool, and component library availability. Hooks-based architecture is clean for data-fetching patterns.
Vite	Webpack, CRA, Turbopack	10–100x faster HMR than Webpack. Native ESM dev server. Simple config. De facto standard for new React projects.
TanStack Query	SWR, Redux Toolkit Query, Zustand for server state	Purpose-built for server state: caching, deduplication, background refetch, optimistic updates. Eliminates manual loading/error state management.

Tailwind CSS	CSS Modules, styled-components, MUI	Utility-first = fast iteration. No CSS file bloat. PurgeCSS keeps bundle small. No runtime overhead (unlike CSS-in-JS).
Recharts	Chart.js, D3, Victory	React-native charting library (composable components). Good for the chart types we need (bar, line, pie for dashboard). D3 is too low-level for this scope.
Axios	fetch API	Request/response interceptors for auth token injection. Automatic JSON parsing. Better error handling than raw fetch.

10.3 Infrastructure

Choice	Alternative Considered	Why This One
ECS Fargate	EC2, Lambda, App Runner, EKS	Serverless containers = no instance management. Right-sized for a long-running Express server (Lambda cold starts would hurt latency). EKS is overkill (Kubernetes for a single service). App Runner is simpler but less configurable (no VPC placement, limited health checks).
Aurora PostgreSQL Serverless v2	RDS PostgreSQL, Supabase, PlanetScale	Auto-scales ACUs based on load (0.5→4). No manual instance sizing. Multi-AZ by default. Serverless v2 has fast scale-up (<1s) vs v1 (30s+). Cost-efficient for variable workloads. Supabase is a good dev option but less control in production.
CloudFront + S3	Vercel, Netlify, Amplify	Full control over caching behavior. API proxy (`/api/*`) eliminates CORS issues. S3 versioning for rollback. OAC (Origin Access Control) restricts S3 to CloudFront-only access. Vercel/Netlify would simplify but lose control and add vendor lock-in.
ALB	NGINX, API Gateway	Native integration with ECS. Health checks, target group draining. HTTP/2 support. API Gateway adds unnecessary complexity and cost for a REST API that doesn't need throttling/API keys (auth is JWT-based).
Terraform	CloudFormation, CDK, Pulumi	Cloud-agnostic IaC (portable if we move providers). HCL is declarative and readable. Strong state management. CDK generates CloudFormation (AWS-locked). Pulumi requires programming language overhead.
Secrets Manager	Parameter Store, .env files, Vault	Native ECS integration (secrets injected as env vars at task launch). Automatic rotation support. Parameter Store is cheaper but lacks rotation. .env files in containers are a security anti-pattern.
CloudWatch	Datadog, Grafana Cloud, ELK Stack	Native AWS integration. Zero setup for ECS/ALB/RDS metrics. Alarms → SNS → email. Dashboard for ops

		visibility. Datadog is better but $15+/host/month. For MVP, CloudWatch is sufficient.

10.4 Why NOT Microservices

This is a **modular monolith**, not microservices. Here's why:

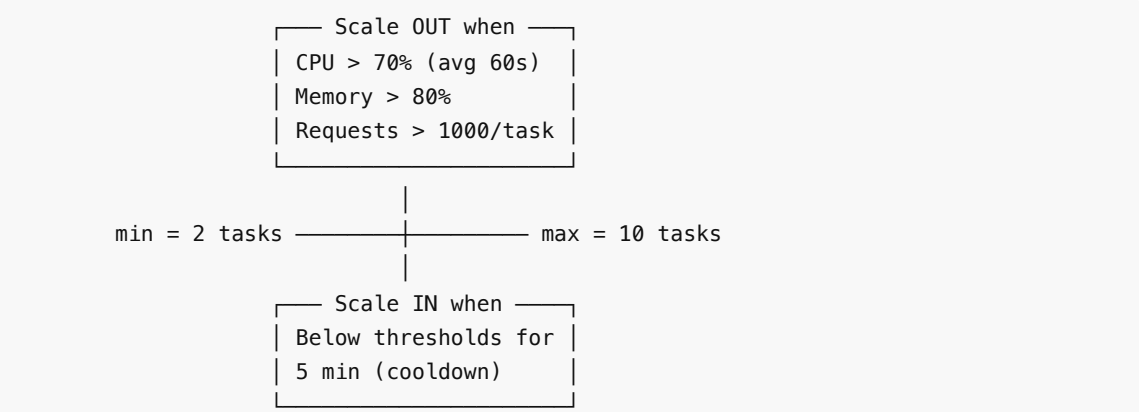
Concern	Monolith (our choice)	Microservices
Deployment complexity	1 Docker image, 1 ECS service	N images, N services, service mesh
Data consistency	Single DB, ACID transactions	Distributed transactions / sagas
Dev team size	Solo / small team	Requires per-service ownership
Latency	In-process function calls	Network hops between services
Debugging	Single log stream	Distributed tracing required

The service layer (`src/services/`) is the seam. If auth becomes a bottleneck (stress test showed this), we can extract `AuthService` into its own microservice by putting an HTTP adapter in front of it — without changing the rest of the codebase.

---

11. Scaling Strategy

11.1 Horizontal Scaling (ECS Auto-Scaling)



11.2 Database Scaling (Aurora Serverless v2)

```
Load increases → Aurora auto-scales ACUs
  0.5 ACU → 1 ACU → 2 ACU → 4 ACU (max)
  Response time: < 1 second to scale

Production: 2 instances (writer + reader replica)
  └── Read-heavy queries (dashboard, reports) can be
      directed to reader replica via connection string
```

11.3 What Scales When

Bottleneck	Current Limit	Scale Action
CPU (bcrypt)	0.25 vCPU saturates at ~25 concurrent auth	Increase `backend_cpu` to 1024 (1 vCPU) or add more tasks
Concurrent connections	Prisma default pool = 10 per task	Increase pool size or add PgBouncer
Dashboard query speed	Fine at 1K users, ~200ms	Add materialized views or Redis cache at 10K+ users
AI API calls	OpenAI rate limits	Queue with Bull/Redis, process async
CSV import size	2 MB in-memory	Stream parsing for large files
Frontend	CloudFront handles global scale	Already solved by CDN

---

12. Monitoring & Observability

12.1 CloudWatch Alarms

Alarm	Threshold	Action
ECS CPU > 85%	3 min sustained	SNS → email alert
ECS Memory > 85%	3 min sustained	SNS → email alert
ALB 5xx errors > 10	In 5 min window	SNS → email alert
ALB latency > 2s	3 min sustained	SNS → email alert
RDS CPU > 80%	3 min sustained	SNS → email alert

12.2 CloudWatch Dashboard

4-panel dashboard (auto-provisioned by Terraform):

1. **ECS CPU & Memory** — utilization over time
2. **ALB Requests & Latency** — throughput + response times
3. **ALB HTTP Errors** — 4xx and 5xx counts
4. **RDS Aurora** — CPU, connections, serverless capacity (ACU)

12.3 Logging Pipeline

```
Application (Winston) → stdout (JSON) → CloudWatch Logs (30-day retention)
                                                  │
                                                  ▼
                                        Log Insights queries
                                        (ad-hoc debugging)
```

---

13. Security Design

Layer	Implementation
Authentication	bcrypt (12 rounds) password hashing. JWT in HttpOnly + Secure + SameSite=Strict cookies.
Authorization	Every query is scoped by `userId` — users can only access their own data. Enforced at repository layer.
Input Validation	Zod schemas at API boundary. Rejects invalid data before it reaches services.
SQL Injection	Prisma ORM — all queries are parameterized. No raw SQL strings.
XSS	HttpOnly cookies (JS can't read tokens). React auto-escapes rendered content.
CSRF	SameSite=Strict cookies + CORS origin whitelist.
Network Isolation	ECS tasks in private subnets. DB in private subnets. Only ALB in public subnets. NAT Gateway for outbound.
Secrets	AWS Secrets Manager — injected at container start. Never in code, env files, or Docker images.
Encryption	Aurora storage encrypted at rest. CloudFront enforces HTTPS (redirect HTTP→HTTPS).
Audit Trail	Append-only AuditLog table for all mutations. IP address captured.
File Upload	multer: memory storage, 2 MB limit, .csv extension whitelist.
AI Privacy	Only pre-aggregated monthly summaries sent to OpenAI. Raw transactions never leave the server.

---

14. Stress Test Findings

Test Setup

- **Tool:** Grafana k6 v1.7.1
- **Infrastructure:** AWS App Runner (0.25 vCPU, 0.5 GB RAM) + Supabase PostgreSQL
- **Scenarios:** Smoke (3 VUs, 30s) → Ramping (0→100 VUs, 5 min) → Constant API barrage (30 req/s, 2 min)

Results Summary

Scenario	Outcome
Smoke (3 VUs)	100% pass, 4.6 req/s, p95 = 266ms
Full Stress (100 VUs)	58% HTTP failures, p95 = 60s (timeout)

Root Cause: bcrypt + Tiny Instance

```
bcrypt hash (12 rounds) ≈ 250ms on 1 CPU
On 0.25 vCPU with 50 concurrent auth requests:
```

```
→ Queue depth = 50 × 250ms / 0.25 = 50 seconds
→ Exceeds 60s timeout → cascading failure
```

Auth blocks all downstream operations (no token → all authenticated endpoints fail).

Capacity Table

Concurrent Users	Performance
1–3	Excellent (< 300ms p95)
5–10	Good (< 1s p95)
10–25	Degraded
25–50	Poor (50%+ failures)
50–100	Unusable

What's NOT the Bottleneck

- **Database:** Queries are fast when CPU is available (median 400ms for reads)
- **Networking:** Health endpoint = 100% even under load
- **CloudFront:** CDN layer is healthy

Fix Path

1. **Immediate:** Scale `backend_cpu` from 256 → 1024 (4x improvement, ~$30/month more)
2. **Medium term:** Add Redis for session caching, rate-limit auth endpoints
3. **Long term:** Extract auth into separate high-CPU service

---

15. Trade-offs & What I'd Do Differently at Scale

Current Trade-offs (Accepted for MVP)

Trade-off	Why Accepted
Dashboard computed on-demand (no cache)	Always fresh data, simpler architecture. Add Redis or materialized views when query latency exceeds 500ms at scale.
In-process cron (node-cron)	No external scheduler dependency. Replace with EventBridge when we need guaranteed delivery and multi-instance coordination.
Single DB for everything	ACID simplicity. AuditLog and Transactions in the same DB means simple joins. Split when storage exceeds 100 GB or read/write patterns diverge significantly.
Monolith	In-process calls are faster and simpler than network hops. Extract services along the seams (`AuthService`, `SavingsAIService`) only when team/scaling demands it.
No message queue	CSV import and AI calls are synchronous. Fine for 2 MB files and ~1.7s LLM calls. Add SQS/Bull when imports exceed 10 MB or AI latency exceeds 5s.

What Changes at 100K Users

Component	Current	At Scale
Dashboard	On-demand SQL	Redis cache (1 min TTL) or materialized views
Auth	bcrypt in Express	Separate auth service or managed auth (Cognito)
AI Planner	Synchronous	Async via SQS → Lambda, webhook when ready
Audit Log	Same DB	Separate time-series DB (TimescaleDB) or S3 + Athena
Scheduler	node-cron	AWS EventBridge + SQS
CSV Import	In-memory (2 MB)	Streaming via S3 pre-signed upload → Lambda processor
Search	SQL LIKE queries	Elasticsearch for transaction search
DB	Single Aurora cluster	Read replicas, connection pooling (PgBouncer)
Monitoring	CloudWatch	Datadog for APM, distributed tracing

Interview Talk Track (10 Minutes)

> ***Minute 0–1 (Problem & Requirements):***
> *"FinPilot is a personal finance app I built end-to-end. Users track multi-currency transactions, set budgets, import bank CSVs, and get AI-driven savings advice. The hard requirements were financial accuracy — numbers can't change retroactively — and data privacy for AI features."*

> ***Minute 1–3 (Architecture):***
> *"Two separate deployments: a React SPA on CloudFront/S3 and a Node.js API on ECS Fargate with Aurora PostgreSQL Serverless. The backend is a modular monolith with three layers — Routes parse HTTP, Services contain business logic, Repositories talk to the DB via Prisma. This separation means I can unit-test services by mocking repos, and swap DB queries without touching business logic."*

> ***Minute 3–5 (Key Design Decisions):***
> *"Three decisions I'd highlight. First, write-time currency conversion: when a user logs a EUR transaction, I immediately convert to their base currency using today's rate and store both values. This means aggregations are instant and March's report shows the same numbers in June. Second, the AI planner only receives pre-aggregated monthly category totals — never raw transactions — for privacy and cost efficiency (~$0.002/call). Third, JWT in HttpOnly cookies for stateless auth that scales horizontally without a session store."*

> ***Minute 5–7 (Database & Storage):***
> *"9 tables in PostgreSQL. Transactions are the core — Decimal(15,2) for money to avoid floating-point errors, composite indexes for every dashboard query path. AuditLog is append-only for compliance. At 1,000 users, I estimated ~128 MB/month growth, with AuditLog being 78% of that — so my clearing strategy archives audit data to S3 after 1 year and moves transactions older than 2 years to cold storage."*

> ***Minute 7–8 (Scaling & Bottlenecks):***
> *"I ran k6 stress tests and found that bcrypt on a 0.25 vCPU Fargate task was the bottleneck — auth requests queued up and cascaded failures at 25+ concurrent users. The fix is straightforward: scale to 1*

> *vCPU, and longer-term, extract auth into a separate service. The database wasn't the bottleneck. ECS auto-scales 2→10 tasks based on CPU, memory, and request count."*

> ***Minute 8–9 (Infrastructure & Security):***
> *"Fully IaC with Terraform. CloudFront handles TLS termination and SPA routing, proxies /api/* to the ALB with zero caching. Backend runs in private subnets, DB is private, only the ALB is public-facing. Secrets are in AWS Secrets Manager, injected at container launch. CloudWatch alarms for CPU, memory, 5xx errors, and latency notify via SNS."*

> ***Minute 9–10 (Trade-offs & Evolution):***
> *"This is an MVP architecture. I made deliberate trade-offs: no Redis cache (dashboard queries are fast enough on SQL), no message queue (CSV import and AI calls are fast enough synchronously), and a monolith (because in-process calls beat network hops for a small team). I know exactly where the seams are. If we hit 100K users, I'd add Redis for the dashboard, move to async AI processing with SQS, and extract auth into its own service."*

Document generated from actual codebase analysis. Every number, endpoint, and configuration is sourced from the live code, Terraform configs, and stress test results.